

Password: PiXaSWQqHq

In Linux, `ltrace` is a command-line diagnostic tool that shows you the exact interactions between a program and the shared libraries it relies on. It displays the name of the function being called, the arguments passed into it, and the return value.

```
-r-sr-x--- 1 leviathan2 leviathan1 7452 Aug 26 2019 check
```

S means setuid as leviathan2

`ltrace ./check`

```
__libc_start_main(["./check"] <unfinished ...>
printf("password: ")                                = 10
getchar(0xf7fc5310, 0xf7fc3000, 0x786573, 0x646f67password: dd
)                                                    = 100
getchar(0xf7fc5310, 0xf7fc3064, 0x786573, 0x646f67)    = 100
getchar(0xf7fc5310, 0xf7fc6464, 0x786573, 0x646f67)    = 10
strcmp("dd\n", "sex")                                = -1
puts("Wrong password, Good Bye ... "Wrong password, Good Bye ...
)                                                    = 29
```

Password: ERJ9jTYWXE

A **Link** is a symbolic pointer to a file to allow access to the file from other folders. Instead of creating a copy that, when edited, will be different to the original, a link points to the original.

Quick Comparison

Feature	Symbolic Link (ln -s)	Hard Link (ln)
What it is	A file containing a text path	A duplicate pointer to the same data

Feature	Symbolic Link (ln -s)	Hard Link (ln)
If original is deleted	Link breaks (useless)	Link still works and holds the data
Can link to folders?	Yes	No
Works across drives?	Yes	No

. The Permission Check (access)

Plaintext

```
access("/tmp/cmon/test file.txt", 4) = 0
```

The program uses the `access()` system call to check if your current user has read permissions (4 stands for `R_OK`) for the file `/tmp/cmon/test file.txt`. Because you created or have access to this specific file, `access()` returns 0 (Success). The program thinks everything is safe.

2. The String Creation (sprintf)

Plaintext

```
sprintf(..., "/bin/cat %s", "/tmp/cmon/test file.txt")
```

The program constructs a shell command string using `snprintf`. It takes your input and drops it straight into a template: `/bin/cat %s`. Crucially, **the developer forgot to put quotes around %s**. The resulting string stored in memory is exactly: `/bin/cat /tmp/cmon/test file.txt`

3. The Execution (system)

Plaintext

```
system("/bin/cat /tmp/cmon/test file.txt")
```

The `system()` function takes that string and passes it directly to a Unix shell (`/bin/sh`).

Because there are no quotes inside the execution string, the shell interprets the **space** in `"test file.txt"` as an argument separator. Instead of telling `cat` to open one file named `test file.txt`, the shell treats it as a command to open **two separate files**:

1. `/tmp/cmon/test`
2. `file.txt`

Why this creates a vulnerability

When you ran the binary normally:

Bash

```
leviathan2@leviathan:~$ ./printfile /tmp/cmon/"test file.txt"  
PiEpxxknZH  
cat: file.txt: No such file or directory
```

1. `access()` verified you had permission to read `/tmp/cmon/test file.txt`.
2. `setreuid(12002, 12002)` dropped/set privileges to the `leviathan3` level (the owner of the `printfile` binary).
3. `system()` executed `cat /tmp/cmon/test file.txt` with those elevated privileges.
4. `cat` successfully opened and printed the contents of `/tmp/cmon/test` (which leaked the password `PiEpxxknZH`), and then complained that it couldn't find the second file (`file.txt`).

By creating a file with a space in its name, you bypassed the `access()` check while forcing the underlying `cat` command to read an entirely different file (`/tmp/cmon/test`) that you normally wouldn't have permission to read directly!

Password: PiEpxxknZ

When its being traced suid binaries blocks the privilege.

ltrace ./level3: does not work(user still leviathan3)

./level3: works(user now leviathan4 and has access to /etc/leviathan_pass/leviathan4

Password: XlyBbRwAPt

Translating binary code

By pure code:

echo

```
"0100001001110101011000100011100101100111  
0101101000110011010000100100011101010101  
00001010" | perl -lpe '$_=pack("B*",$_)'
```

by the file itself(simpler):

```
./bin | perl -lape '$_=pack"(B8)*",@F'
```

The -a flag splits the 8-bits by spaces

- "B8"* tells Perl to look at every element in the array @F and treat each individual element as an exact **8-bit byte** string.
- By passing @F, it processes every space-separated binary word produced by ./bin and joins them into a single string.

./bin | tr -d ' ' | perl -lpe '\$_=pack("B*", \$_)' this works too(gets rid of spaces)

Password: Bub9gZ3BGU

```
In -s /etc/leviathan_pass/leviathan6 /tmp/file.log
```

Password: JRGj9iWNOB

Method 1: brute force

for i in {0000..9999}; do ./leviathan6 \$i; done | grep -v "Wrong" enters the correct 4 digit and enters the shell, which then you can run cat /etc/leviathan_pass/leviathan7

method 2: assembly(?)

Instead, we can get a little more into reverse engineering and look into the code with gdb.

First, we start it:

```
1 gdb --args leviathan6 0000
```

Then we look at the assembly code of the main function. We need to find the line where the digits are compared:

```
1 (gdb) disassemble main
2 Dump of assembler code for function main:
3   0x080491d6 <+0>:    lea    0x4(%esp),%ecx
4   0x080491da <+4>:    and    $0xffffffff0,%esp
5   0x080491dd <+7>:    push  -0x4(%ecx)
6   0x080491e0 <+10>:   push  %ebp
7   0x080491e1 <+11>:   mov    %esp,%ebp
8   0x080491e3 <+13>:   push  %ebx
9   0x080491e4 <+14>:   push  %ecx
10  0x080491e5 <+15>:   sub    $0x10,%esp
11  0x080491e8 <+18>:   mov    %ecx,%eax
12  0x080491ea <+20>:   movl   $0x1bd3,-0xc(%ebp)
13  0x080491f1 <+27>:   cmpl   $0x2, (%eax)
14  0x080491f4 <+30>:   je     0x8049216 <main+64>
15  0x080491f6 <+32>:   mov    0x4(%eax),%eax
16  0x080491f9 <+35>:   mov    (%eax),%eax
17  0x080491fb <+37>:   sub    $0x8,%esp
18  0x080491fe <+40>:   push  %eax
19  0x080491ff <+41>:   push  $0x804a008
20  0x08049204 <+46>:   call   0x8049050 <printf@plt>
21  0x08049209 <+51>:   add    $0x10,%esp
22  0x0804920c <+54>:   sub    $0xc,%esp
23  0x0804920f <+57>:   push  $0xffffffff
24  0x08049211 <+59>:   call   0x8049090 <exit@plt>
25  0x08049216 <+64>:   mov    0x4(%eax),%eax
26  0x08049219 <+67>:   add    $0x4,%eax
27  0x0804921c <+70>:   mov    (%eax),%eax
28  0x0804921e <+72>:   sub    $0xc,%esp
29  0x08049221 <+75>:   push  %eax
30  0x08049222 <+76>:   call   0x80490b0 <atoi@plt>
31  0x08049227 <+81>:   add    $0x10,%esp
32  0x0804922a <+84>:   cmp    %eax,-0xc(%ebp)
33  0x0804922d <+87>:   jne    0x804925a <main+132>
34  0x0804922f <+89>:   call   0x8049060 <geteuid@plt>
35  0x08049234 <+94>:   mov    %eax,%ebx
36  0x08049236 <+96>:   call   0x8049060 <geteuid@plt>
37  0x0804923b <+101>:  sub    $0x8,%esp
38  0x0804923e <+104>:  push  %ebx
39  0x0804923f <+105>:  push  %eax
40  0x08049240 <+106>:  call   0x80490a0 <setreuid@plt>
41  0x08049245 <+111>:  add    $0x10,%esp
```

```

42 0x08049248 <+114>: sub    $0xc,%esp
43 0x0804924b <+117>: push   $0x804a022
44 0x08049250 <+122>: call   0x8049080 <system@plt>
45 0x08049255 <+127>: add    $0x10,%esp
46 0x08049258 <+130>: jmp    0x804926a <main+148>
47 0x0804925a <+132>: sub    $0xc,%esp
48 0x0804925d <+135>: push   $0x804a02a
49 0x08049262 <+140>: call   0x8049070 <puts@plt>
50 0x08049267 <+145>: add    $0x10,%esp
51 0x0804926a <+148>: mov    $0x0,%eax
52 0x0804926f <+153>: lea    -0x8(%ebp),%esp
53 0x08049272 <+156>: pop    %ecx
54 0x08049273 <+157>: pop    %ebx
55 --Type <RET> for more, q to quit, c to continue without paging-- c
56 0x08049274 <+158>: pop    %ebp
57 0x08049275 <+159>: lea    -0x4(%ecx),%esp
58 0x08049278 <+162>: ret
59 End of assembler dump.

```

Looking at the code, we can see line '+106' calls 'setreuid@plt' and line '122' calls 'system@plt'. The names sound, like the user id, is changed and a shell command is executed. We can check this assumption, by looking into the [Linux Manual Page](#). We find this is exactly, what is happening. So now we just need to follow the program flow, to see, how we get to this part of the code. We will find that the jump in line '+87' would lead to skipping that part because it jumps to line '+132'. This must be, where the code is checked and if it is not equal, it skips the for us important part. another way to see this is the line '+76', where the 'atoi' function is called (converts string to integer - assume it's our input). So we need to check what is compared (line '+84').

```

1 (gdb) break *0x0804922a
2 Breakpoint 1 at 0x804922a
3 (gdb) run
4 Starting program: /home/leviathan6/leviathan6 0000
5 [Thread debugging using libthread_db enabled]
6 Using host libthread_db library "/lib/x86_64-linux-
7 gnu/libthread_db.so.1".
8
9 Breakpoint 1, 0x0804922a in main ()
10 (gdb) info registers
11 eax                0x0

```

12	ecx	0x0	0
13	edx	0xffffd71e	-10466
14	ebx	0xf7fab000	-134565888
15	esp	0xffffd4c0	0xffffd4c0
16	ebp	0xffffd4d8	0xffffd4d8
17	esi	0xffffd5a4	-10844
18	edi	0xf7ffc80	-134231168
19	eip	0x804922a	0x804922a <main+84>
20	eflags	0x286	[PF SF IF]
21	cs	0x23	35
22	ss	0x2b	43
23	ds	0x2b	43
24	es	0x2b	43
25	fs	0x0	0
26	gs	0x63	99
27	k0	0x0	0
28	k1	0x0	0
29	k2	0x0	0
30	k3	0x0	0
31	k4	0x0	0
32	k5	0x0	0
33	k6	0x0	0
	k7	0x0	0

So looking at the line `0x0804922a <+84>: cmp %eax,-0xc(%ebp)`, we compare the value in the register 'eax', which is 0, so our initial input, with what is stored on the address in register 'ebp' minus 0xc.

First, we calculate the address, and then we look at what is saved at that address in decimal:

```

1 (gdb) print $ebp-0xc
2 $1 = (void *) 0xffffd4cc
3 (gdb) x 0xffffd4cc
4 0xffffd4cc: 0x00001bd3
5 0xffffd4cc: 0x00001bd3
6 (gdb) print/d 0x00001bd3
7 $3 = 7123

```

And we got a 4-digit number, now we just need to test, if it is the correct one.


```

1 leviathan6@leviathan:~$ ./leviathan6 7123
2 $ whoami
3 leviathan7
4 $ cat /etc/leviathan_pass/leviathan7
5 ahy7MaeBo9

```

We got the right code and a shell, as leviathan7. So we can read the password file.

1. Assembly Language Basics (AT&T Syntax)

Assembly is a low-level language bridging the gap between human-readable code and machine code.

- **Structure of a line:** instruction source, destination
- **Key Components:**
 - **Instruction:** The action being performed (e.g., add, mov, cmp, je).
 - **Source/Destination:** Can be a hard-coded value (prefixed with \$), a CPU register (prefixed with %), or a memory address.
- **Examples:**
 - add \$0x4, %eax $\rightarrow$ Adds the value 0x4 to the %eax register.
 - mov (%eax), %eax $\rightarrow$ Moves the value stored *at the memory address* inside %eax into %eax itself.

2. GDB (GNU Debugger) Quick Reference

GDB is a tool used to inspect and reverse-engineer compiled programs.

Core Commands

Command	Shortcut	What it does
<code>gdb --args program params</code>	<i>Terminal command</i>	Starts GDB with your target program and its arguments.
<code>run</code>	<code>r</code>	Starts executing the program from the

Command	Shortcut	What it does
		beginning.
<code>break *0xaddress</code>	<code>b *0x...</code>	Sets a breakpoint at a specific memory address to pause execution.
<code>continue</code>	<code>c</code>	Resumes running the program after hitting a breakpoint.
<code>disassemble function</code>	<code>disas</code>	Displays the assembly code for a specific function (e.g., <code>main</code>).
<code>info registers</code>	<code>i r</code>	Prints the current values stored in all CPU registers.
<code>x 0xaddress</code>	<code>x</code>	Examines and prints the value saved at a specific memory address.

3. Key Concepts to Study Further

To get better at binary levels like Leviathan, look up tutorials on:

- **Registers:** Small, lightning-fast storage locations inside the CPU (like `%eax`, `%ebx`).
- **Memory Addresses:** How the computer flags and finds specific spots in its RAM.

(gdb) `print -d 0x00001bd3`

- `print`: "Hey GDB, calculate or show me this value."

- `/d`: "...and format the output in **d**ecimal (base-10) so it looks like a normal human number."
- `0x00001bd3`: "Here is the hexadecimal value I want you to translate."

GDB looked at `0x1bd3`, did the math under the hood:

$$\begin{aligned}
 & \$\$ (1 \times 16^3) + (11 \times 16^2) + (13 \times 16^1) + (3 \times 16^0) \\
 & = 4096 + 2816 + 208 + 3 = 7123 \$\$
 \end{aligned}$$

Password: 3zrlkaPTfH